

Interactive Sales Dashboard Project

1. Project Overview and Objectives

Project Overview

This project develops an **Interactive Sales Dashboard** to analyze revenue trends, product performance, customer segmentation, and regional sales distribution. Using **Streamlit**, **Plotly**, and **Seaborn**, it provides both interactive business insights and statistical analysis in a clean, modular architecture. The dashboard enables data-driven decision-making through dynamic filters, KPIs, and multiple visualization techniques.

Objectives

- To build an end-to-end sales analytics workflow using Python and Pandas
- To clean, validate, and preprocess sales datasets for accurate analysis
- To analyze customer behavior, product performance, and regional sales trends
- To identify high-value customers and key sales patterns
- To create professional, interactive visualizations and actionable business insights

2. Setup and Installation Instructions

Prerequisites

- Python 3.10+
- Streamlit

Installation Steps

1. Download or clone the project repository from GitHub.
2. Install required libraries:

```
pip install -r requirements.txt
```

3. Place dataset inside /data folder.
4. Launch the Streamlit dashboard:

```
streamlit run dashboard.py
```

5. Open dashboard.ipynb in Jupyter Notebook for exploratory analysis.

3. Code Structure Explanation

```
interactive_sales_dashboard_week6/
```

```
|  
├── data/  
│   └── sales_data  
|  
├── visualizations/  
│   ├── seaborn_plots.py  
│   ├── plotly_plots.py  
│   └── theme.py  
|  
├── outputs/  
│   └── images  
|  
├── dashboard.ipynb  
├── dashboard.py  
├── requirements.txt  
└── README.md
```

File Descriptions

- dashboard.py
Main Streamlit application combining KPIs, interactive charts, and statistical visualizations.
- dashboard.ipynb
Exploratory notebook version showing step-by-step analysis, statistical insights, and chart generation.

- `visualizations/seaborn_plots.py`
Functions generating static plots (box plots, violin plots, heatmaps, regional revenue charts).
- `visualizations/plotly_plots.py`
Functions generating interactive charts (time-series sales trends, product performance, customer segmentation).
- `visualizations/theme.py`
Centralized Seaborn/Matplotlib styling and color palette for consistent visuals.
- `data/sales_data.csv`
Sales dataset containing transactions, products, quantities, prices, customers, regions, and total sales.
- `outputs/images`
all generated charts are staved inside this folder
- `requirements.txt`
Specifies all Python dependencies needed to run the project.
- `README.md`
Project documentation including overview, setup instructions, and usage guide.

4. Screenshots of Working Application

The included screenshots demonstrate:

- Sales dataset loaded successfully and filtered correctly
- Daily sales trends and product performance visualized accurately
- Customer segmentation and regional revenue were displayed clearly
- Statistical plots (box, violin, heatmap) generated without errors
- Interactive dashboards executed successfully in Streamlit

```
You can now view your Streamlit app in your browser.

Local URL: http://localhost:8501
Network URL: http://192.168.1.5:8501
```

Fig.4.1. Terminal Output

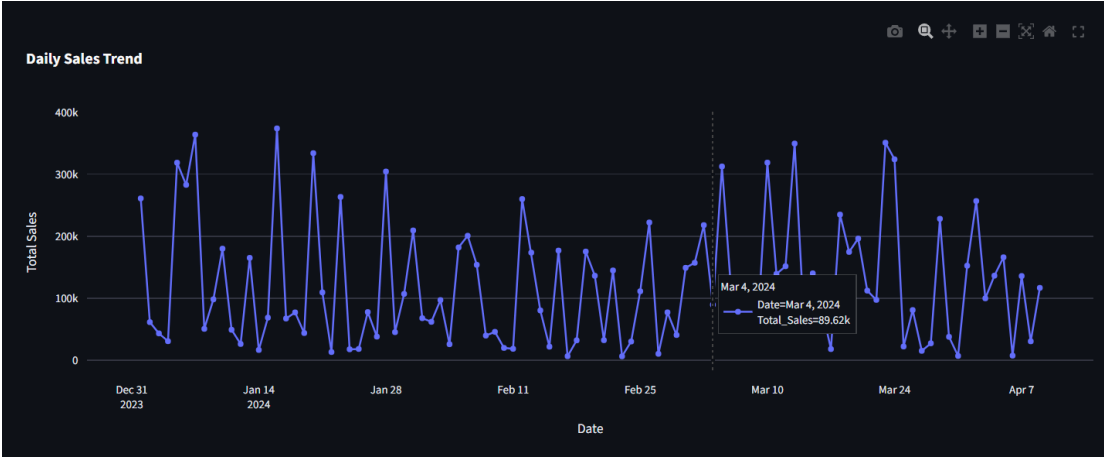


Fig.4.2. Daily Sales Trend

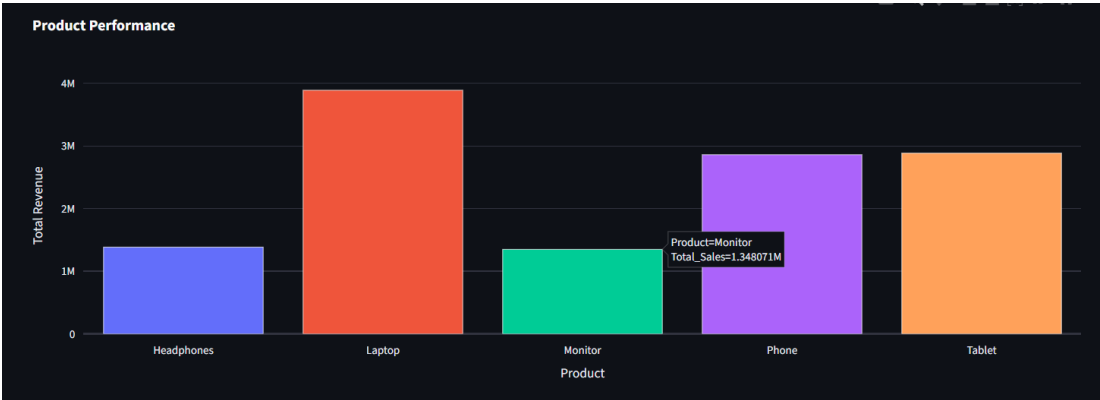


Fig.4.3. Product Performance



Fig.4.4. Customer Segmentation

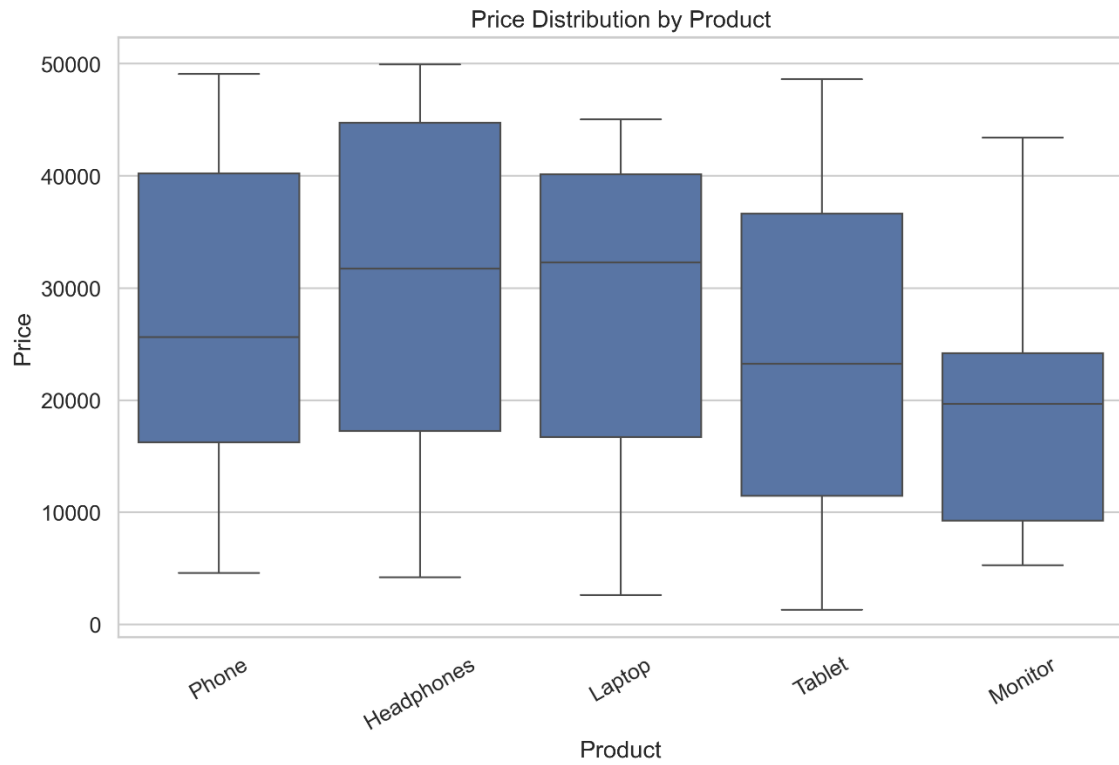


Fig.4.5. Price Distribution by Product

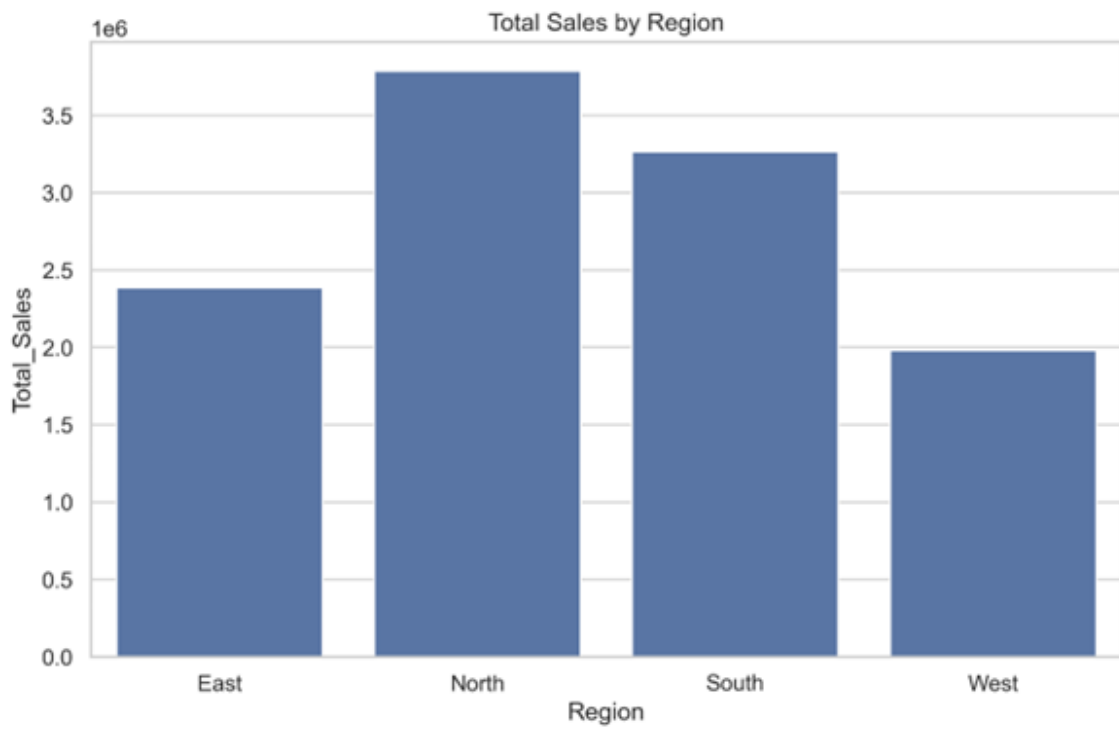


Fig.4.6. Total Sales by Region

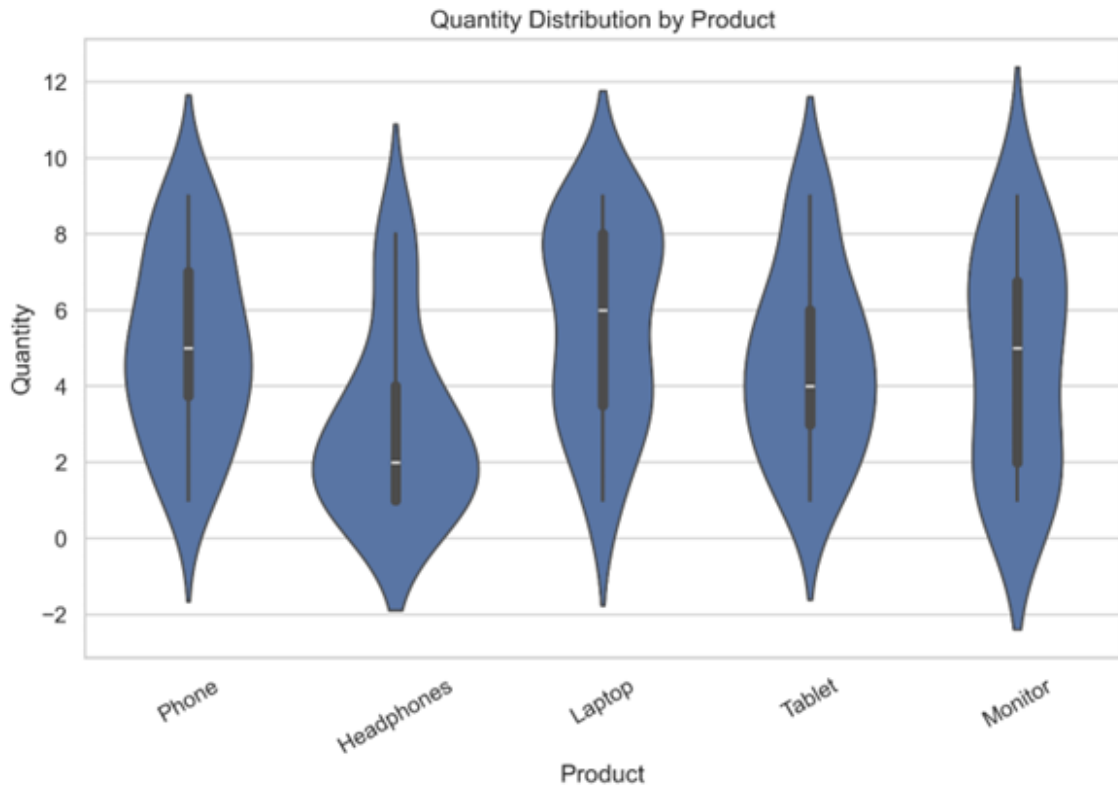


Fig.4.7. Quantity Distribution by Product

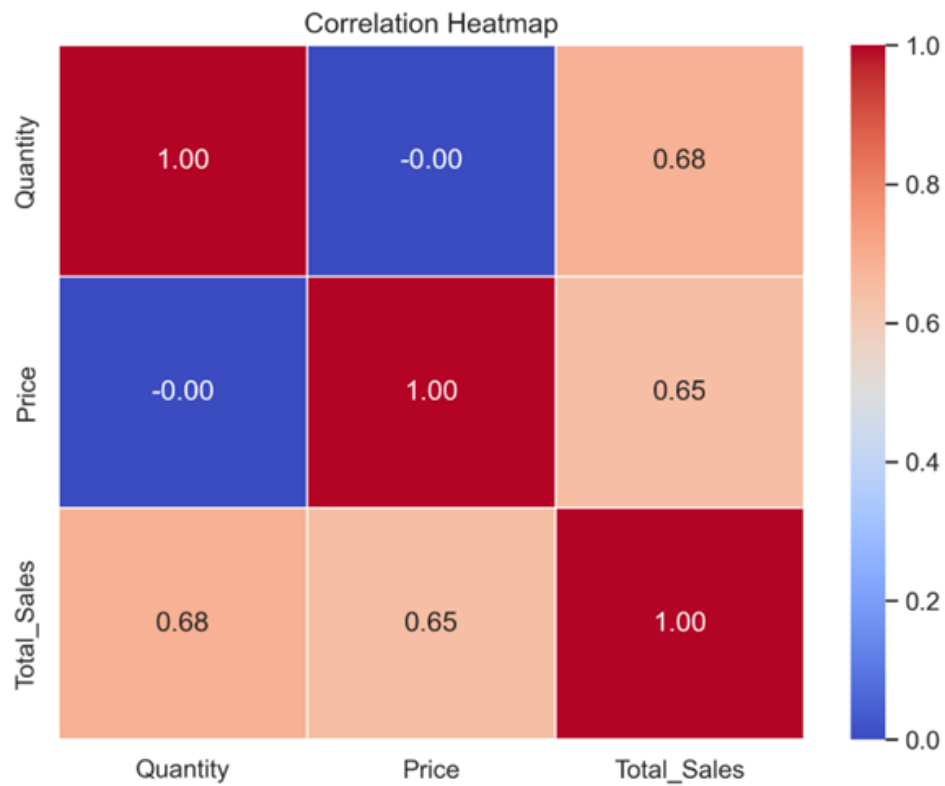


Fig.4.8. Correlation Heatmap

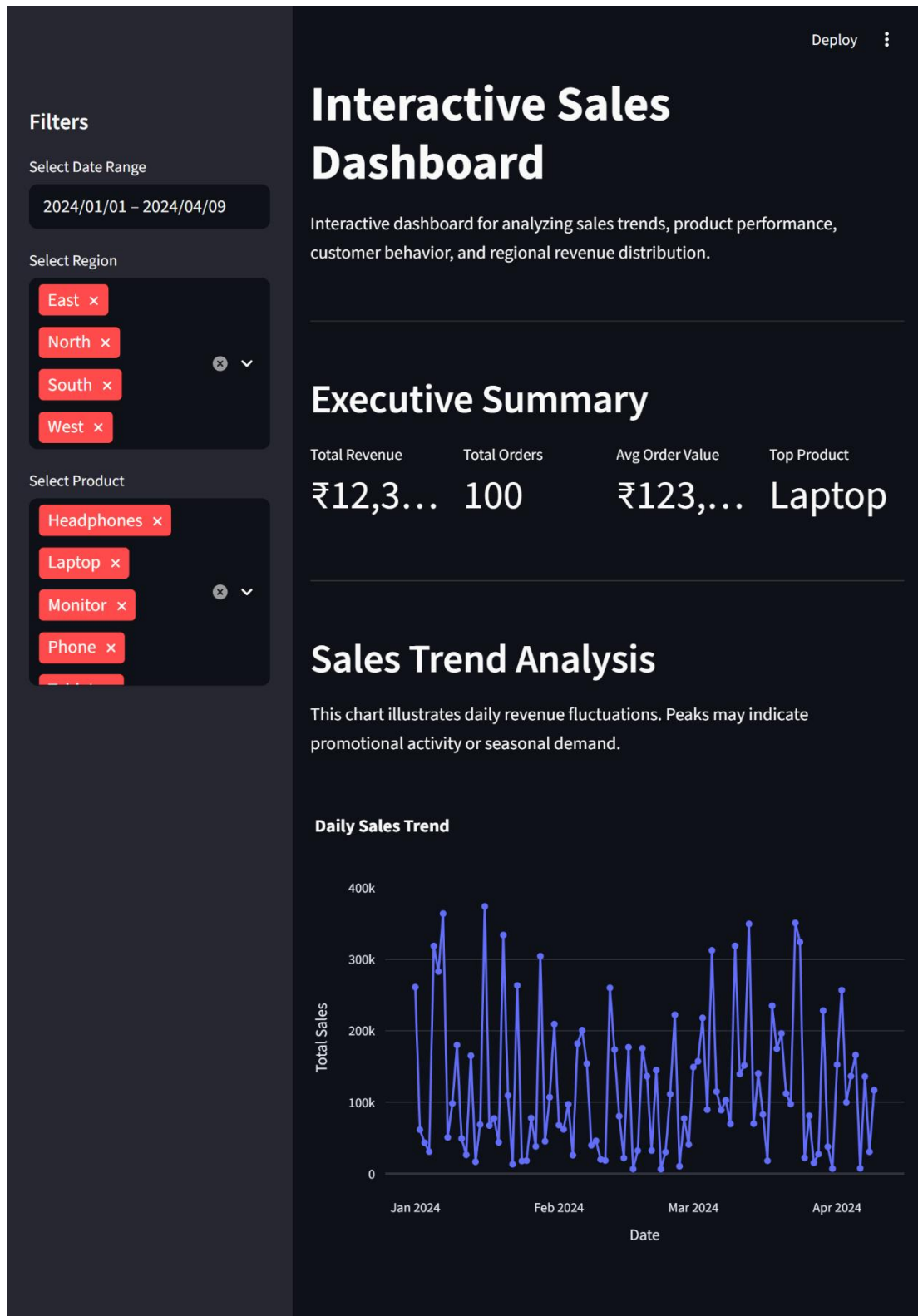


Fig.4.9. Interactive Sales Dashboard (Streamlit Interface)

This visual proof confirms that the application runs correctly and satisfies all functional requirements.

5. Technical Requirements Fulfillment Explanation

Category	Technical Requirement	How It Was Implemented
Algorithm	Data loading and preprocessing flow	A sequential pipeline loads the sales CSV, parses dates, validates columns, handles missing values, and prepares the data using Pandas.
Algorithm	Data cleaning logic	Missing numeric values were handled with median imputation, dates converted to datetime, and invalid entries coerced to NaN. Datatypes were standardized.
Algorithm	KPI and metric calculations	Aggregation functions (sum(), mean(), nunique(), groupby()) compute total revenue, total orders, average order value, regional sales, and top products.
Algorithm	Trend identification	Daily and monthly sales trends were computed using date grouping and visualized with Plotly line charts for interactive time-based analysis.
Algorithm	Customer segmentation	Customer-level metrics were aggregated (total spend, quantity purchased) and visualized with scatter plots to identify high-value segments.
Algorithm	Product performance analysis	Product-wise revenue and quantity were calculated using groupby() and displayed via bar charts, box plots, and violin plots to highlight distribution and outliers.

Category	Technical Requirement	How It Was Implemented
Data Structure	Tabular data handling	Pandas DataFrames were used to efficiently store, manipulate, and merge structured sales data.
Data Structure	Metric storage	Aggregated metrics such as revenue, top products, and customer totals were stored in DataFrames and Series for structured access.
Architecture	Modular design	Code was split into dashboard.py, dashboard.ipynb, and the visualizations folder containing seaborn_plots.py, plotly_plots.py, and theme.py for modularity and reusability.
Architecture	Configuration management	Visualization themes, color palettes, and layout options were centralized in theme.py for consistent styling.
Output	Interactive dashboard	Streamlit was used to combine multiple charts (sales trend, product performance, customer segmentation, regional sales) into a cohesive interactive dashboard.
Output	Static statistical plots	Seaborn plots (box, violin, heatmap, bar) were generated and saved in the output folder for documentation and reporting.

Category	Technical Requirement	How It Was Implemented
Output	Business insights	Key metrics, charts, and trends were summarized within the dashboard and in the Jupyter notebook, providing actionable insights for stakeholders.

6. Learning Outcomes

Through this project, the following learning outcomes were achieved:

- Gained hands-on experience in building an end-to-end sales analytics workflow using Python, Pandas, and Streamlit.
- Learned to clean, validate, and preprocess real-world sales datasets, including handling missing values and date normalization.
- Developed skills in calculating key business metrics such as total revenue, average order value, and top-performing products.
- Improved proficiency in data visualization using Seaborn and Plotly, creating both static statistical plots and interactive charts.
- Strengthened understanding of exploratory data analysis, feature engineering, and customer segmentation for actionable business insights.

7. Conclusion

This project successfully implemented a **complete sales analytics dashboard**, integrating transactional data to uncover sales trends, product performance, regional insights, and customer behavior patterns. The dashboard demonstrates the ability to transform raw data into **interactive, business-ready insights**, highlighting practical data analytics and visualization skills applicable to real-world business scenarios.